

Manual Maestro: Explicación de Predicciones en Redes Neuronales Convolucionales mediante Gradientes Integrados

1. Introducción: La Imperativa de la Interpretabilidad en 2025

En el umbral del año 2026, la inteligencia artificial ha dejado de ser una curiosidad académica para convertirse en la infraestructura invisible que gobierna decisiones críticas en medicina, finanzas, transporte y justicia. Las Redes Neuronales Convolucionales (CNN), arquitecturas que imitan la corteza visual biológica, han alcanzado niveles de precisión que frecuentemente superan la capacidad humana en tareas de clasificación de imágenes. Sin embargo, este rendimiento sobrehumano conlleva un costo significativo: la opacidad. A medida que los modelos aumentan en profundidad y complejidad —pasando de las arquitecturas ResNet clásicas a los modernos Vision Transformers y EfficientNets— la relación entre la entrada (una imagen) y la salida (una predicción) se vuelve cada vez más inescrutable.

Este fenómeno, conocido como el problema de la "caja negra", ha dejado de ser un inconveniente técnico para convertirse en un riesgo sistémico. ¿Cómo puede un radiólogo confiar en un diagnóstico de cáncer asistido por IA si el modelo no puede señalar qué tejido específico activó la alerta? ¿Cómo puede un auditor certificar un sistema de conducción autónoma si no entiende por qué el vehículo decidió frenar bruscamente en una autopista despejada? La respuesta de la comunidad científica y regulatoria ha sido el desarrollo y estandarización de la Inteligencia Artificial Explicable (XAI).

Entre el vasto arsenal de técnicas de XAI desarrolladas en la última década, los **Gradientes Integrados (Integrated Gradients - IG)** han emergido como el estándar de oro para la atribución de características en modelos diferenciables. A diferencia de las heurísticas empíricas que a menudo producen explicaciones visualmente agradables pero matemáticamente infundadas, los Gradientes Integrados se construyen sobre una base teórica sólida derivada de la teoría de juegos cooperativos y el cálculo integral. Esta técnica no solo nos dice *qué* decidió el modelo, sino que cuantifica con precisión axiomática *cuánto* contribuyó cada píxel individual a esa decisión.

Este "Manual Maestro" es el resultado de una investigación exhaustiva y una síntesis crítica de los tutoriales, papers y documentación técnica mejor valorados entre 2024 y 2025. Su propósito es trascender la fragmentación actual de la información —donde la teoría

matemática, la implementación de código y la configuración del sistema operativo a menudo viven en silos separados— para ofrecer una guía unificada y exhaustiva.

A lo largo de este documento, diseccionaremos la teoría subyacente utilizando analogías intuitivas diseñadas para desmitificar el cálculo complejo; navegaremos por las complejidades de configurar un entorno de desarrollo de alto rendimiento en Arch Linux, abordando los cambios paradigmáticos en los drivers de NVIDIA y la gestión de paquetes de Python introducidos en 2025; y construiremos, paso a paso, un proyecto práctico robusto que no solo implementa IG, sino que también aborda sus limitaciones mediante técnicas avanzadas como el suavizado de ruido (Noise Tunnel) y la validación cuantitativa.

La revisión de la literatura actual revela una convergencia hacia herramientas específicas. Mientras que el ecosistema de TensorFlow mantiene su relevancia, **Captum**, la biblioteca de interpretabilidad de PyTorch, se ha consolidado como la herramienta hegemónica en la investigación y la industria debido a su flexibilidad, su diseño orientado a objetos y su estricta adherencia a los axiomas teóricos de la atribución.¹ Este manual se centrará principalmente en el flujo de trabajo PyTorch/Captum, reflejando las preferencias de la comunidad de expertos en 2025, aunque se harán referencias comparativas a otras herramientas donde sea pertinente.

Prepárese para sumergirse en la mecánica profunda de las redes neuronales. Al finalizar este informe, no solo sabrá cómo generar un mapa de calor sobre una imagen; comprenderá la dinámica de los gradientes que fluyen a través de las capas ocultas y tendrá la capacidad técnica para auditar, depurar y mejorar sus modelos de visión por computadora con una confianza matemática renovada.

2. Fundamentos Teóricos: De la Intuición al Axioma

Para dominar los Gradientes Integrados, es imperativo primero comprender las deficiencias de sus predecesores. La búsqueda de la explicabilidad en las CNN comenzó con una pregunta simple: "¿Qué parte de la entrada es responsable de la salida?". Las primeras respuestas se basaron en el análisis de sensibilidad local, es decir, en los gradientes simples.

2.1 La Falacia de los Gradientes Simples y el Fenómeno de Saturación

La aproximación más inmediata para explicar una red neuronal es tratarla como una función matemática gigante, $F: \mathbb{R}^n \rightarrow \mathbb{R}$, donde la entrada x es una imagen con n píxeles y la salida es la probabilidad de una clase específica. Si queremos saber la importancia de un píxel x_i , el cálculo nos ofrece una herramienta natural: la derivada parcial.

$$\text{Saliency}(x) = \left| \frac{\partial F(x)}{\partial x} \right|$$

Esta técnica, conocida como mapas de saliencia (Saliency Maps), asume que las

características más importantes son aquellas cuyo cambio provoca la mayor variación en la salida. Si modificar ligeramente el color de un píxel cambia drásticamente la predicción de "gato" a "perro", ese píxel debe ser crucial.

Sin embargo, esta lógica intuitiva colapsa ante una propiedad fundamental de las redes neuronales profundas: la **saturación**. Las redes neuronales dependen de funciones de activación no lineales como Sigmoid, Tanh o ReLU para aprender patrones complejos. Estas funciones tienen regiones donde la salida se "aplana" a pesar de que la entrada siga aumentando.

Imagine una neurona que detecta la presencia de un ojo en una imagen. A medida que la evidencia visual de un ojo aumenta (bordes curvos, iris, pupila), la activación de la neurona crece. Pero una vez que la neurona está "completamente segura" de que hay un ojo, añadir más detalle no aumenta su activación; ha alcanzado su valor máximo (saturación). En esta región saturada, la curva de activación es plana. Y en cálculo, la derivada de una línea plana es cero.

Aquí radica la paradoja: la característica (el ojo) es absolutamente esencial para la predicción, pero el método de gradientes simples le asigna una importancia de cero porque, localmente, la función no está cambiando.

Analogía Profunda: El Escalador y la Meseta

Para visualizar este fenómeno, consideremos la analogía del escalador de montañas.²

Imagine que la función de predicción de la red neuronal ($F(x)$) es una montaña, y la altitud representa la confianza del modelo en una clase (por ejemplo, la probabilidad de que la imagen sea un "Barco de Bomberos").

- **El Valle (Línea Base):** Usted comienza al nivel del mar. Esto representa una imagen sin información (negra o ruido), donde la probabilidad de "Barco de Bomberos" es cercana a cero.
- **La Pendiente (Aprendizaje):** A medida que añade características de la imagen (el casco rojo, el agua, las mangueras), empieza a subir la montaña. En las laderas empinadas, cada paso (cada píxel añadido) aumenta significativamente su altitud. Aquí, los gradientes son altos y los métodos simples funcionan bien.
- **La Cima (Predicción Saturada):** Finalmente, llega a la cima de la montaña. La vista es perfecta; el modelo está 99% seguro de que es un barco de bomberos. La cima es una meseta amplia y plana.

Si aplicamos el método de **Gradientes Simples** en este punto, le preguntamos al escalador: "¿Cuánto contribuye el suelo bajo tus pies a tu altitud actual?". El escalador mira hacia abajo, ve que el suelo es plano (gradiente cero), y responde: "Nada. Si me muevo un metro a la izquierda o a la derecha, mi altitud no cambia. Por lo tanto, el suelo bajo mis pies es irrelevante".

Esta conclusión es absurda. El escalador está a 3,000 metros de altura precisamente porque recorrió todo el camino desde la base. Ignorar la trayectoria y centrarse solo en la pendiente local final es el error fundamental que los Gradientes Integrados vienen a corregir.

2.2 La Solución de los Gradientes Integrados: El Camino Integral

Los Gradientes Integrados (IG) resuelven el problema de la saturación al no mirar solo la pendiente en la cima, sino al considerar **todas las pendientes** a lo largo del camino desde el valle hasta la cima.

Matemáticamente, IG define una atribución para cada característica x_i integrando los gradientes a lo largo de un camino lineal que conecta una **línea base** x' (la ausencia de característica) con la entrada actual x .

La ecuación formal es:

$$IG_i(x) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

Desglosemos esta ecuación componente por componente para entender su elegancia:

1. **El Camino ($x' + \alpha(x - x')$):** Imaginamos una serie de imágenes intermedias generadas al interpolar suavemente desde la línea base hasta la imagen real. El parámetro α varía de 0 a 1. Cuando $\alpha=0$, estamos en la línea base (oscuridad total). Cuando $\alpha=1$, estamos en la imagen original. Los valores intermedios (ej. $\alpha=0.5$) son versiones "fantasmales" o atenuadas de la imagen.
2. **El Gradiente ($\frac{\partial F(\dots)}{\partial x_i}$):** En cada paso del camino (cada valor de α), calculamos el gradiente local. Incluso si el modelo está saturado al final ($\alpha=1$), es muy probable que no lo estuviera en los pasos intermedios (ej. $\alpha=0.3$). Al sumar (integrar) estos gradientes intermedios, capturamos la contribución de la característica antes de que ocurriera la saturación.
3. **El Escalado ($(x_i - x'_i)$):** Finalmente, multiplicamos la suma de gradientes por la "distancia" total que la característica recorrió desde la línea base. Esto asegura que la atribución tenga las mismas unidades que la salida del modelo y respete la magnitud del cambio real.

2.3 Los Axiomas Fundamentales: ¿Por qué confiar en IG?

La superioridad de IG no es solo empírica; es teórica. Sundararajan et al. demostraron que IG es el **único** método de atribución que satisface simultáneamente un conjunto de axiomas deseables para cualquier sistema de explicación.⁵ Analicemos los más críticos.

A. Axioma de Completitud (Completeness)

Este es, sin duda, el pilar más fuerte de IG. Establece que la suma de todas las atribuciones de las características debe ser exactamente igual a la diferencia entre la predicción del modelo para la entrada y la predicción para la línea base.

$$\sum_i IG_i(x) = F(x) - F(x')$$

Analogía del Presupuesto Corporativo:

Imagine un departamento de ventas que ha superado su objetivo anual en \$1 millón (la diferencia $F(x) - F(x')$). El gerente quiere distribuir este excedente como bonos (atribuciones) entre los empleados (las características/píxeles) basándose en su rendimiento. El axioma de completitud garantiza una contabilidad perfecta: si sumamos todos los bonos entregados, el total debe ser exactamente \$1 millón. No puede desaparecer dinero ni aparecer dinero de la nada.

Muchos otros métodos, como los gradientes simples o Grad-CAM, no cumplen esta propiedad. Sería como repartir bonos que suman \$800,000 y dejar \$200,000 sin explicar ("dinero negro"), o repartir \$1.2 millones ("inflación"). IG asegura que cada fracción de la predicción del modelo se contabiliza y se asigna a algún píxel.⁷

B. Axioma de Sensibilidad (Sensitivity)

Este axioma tiene dos partes:

1. Si modificamos una entrada y la salida cambia, esa entrada *debe* recibir una atribución no nula. (Esto soluciona el problema de la saturación descrito anteriormente).
2. Si una entrada no afecta en absoluto a la salida (es una variable "dummy"), debe recibir una atribución de cero.

Este axioma protege contra falsos negativos (decir que algo no importa cuando sí importa) y falsos positivos (decir que algo importa cuando es ruido).

C. Invarianza de Implementación

Dos redes neuronales pueden tener arquitecturas internas muy diferentes (por ejemplo, una usa Convoluciones y otra Transformadores, o diferentes funciones de activación) pero ser matemáticamente equivalentes, es decir, producir siempre la misma salida para la misma entrada.

El axioma de invarianza dicta que el método de explicación debe producir las mismas atribuciones para ambas redes. Esto es crucial porque la explicación debe tratar sobre la relación entrada-salida, no sobre los detalles de fontanería interna del modelo. Los métodos que dependen de inspeccionar capas específicas (como Grad-CAM, que mira la última capa convolucional) violan este axioma, ya que son sensibles a la arquitectura. IG, al basarse en gradientes de extremo a extremo, es agnóstico a la implementación.

2.4 Gradientes Integrados y los Valores de Shapley

Es fundamental mencionar que IG no es una invención aislada, sino una aproximación continua de los Valores de Shapley (Shapley Values), un concepto ganador del Premio Nobel en teoría de juegos. Los Valores de Shapley calculan la contribución marginal promedio de un jugador a través de todas las posibles coaliciones de jugadores.

Calcular los Valores de Shapley exactos para una red neuronal es computacionalmente imposible (NP-hard) debido al número exponencial de combinaciones de píxeles. Gradientes Integrados se puede ver como una forma eficiente de calcular los Valores de Shapley Aumann, una extensión de la teoría para juegos con jugadores continuos, haciendo viable su

3. Configuración del Entorno en Arch Linux: La Frontera del 2025

Para el investigador o ingeniero de ML que busca el máximo rendimiento y control, Arch Linux sigue siendo la plataforma de elección en 2025. Sin embargo, el panorama ha cambiado drásticamente en el último año debido a dos eventos sísmicos: la transición de NVIDIA hacia módulos de kernel abiertos y la aplicación estricta del PEP 668 en la gestión de paquetes de Python. Esta sección es una guía técnica actualizada para navegar estos cambios sin romper su sistema.

3.1 La Revolución de los Drivers NVIDIA: Adiós al Blob Propietario (Casi)

Históricamente, los usuarios de Linux dependían del driver propietario de NVIDIA, una "caja negra" binaria que a menudo causaba conflictos con el kernel. A finales de 2024, NVIDIA, en colaboración con la comunidad, hizo la transición de sus paquetes principales hacia los **Open Kernel Modules** (módulos de kernel abiertos) para las arquitecturas modernas. Esto mejora la integración con el kernel de Linux, facilita la depuración y mejora el soporte para Wayland.¹¹

Tabla 1: Matriz de Selección de Drivers NVIDIA en Arch Linux (2025)

Arquitectura GPU	Serie Comercial	Paquete Recomendado (Pacman)	Estado de Soporte
Blackwell / Hopper	RTX 50xx, H100	nvidia-open-dkms	Nativo/Obligatorio. Soporte completo GSP.
Ada Lovelace / Ampere	RTX 40xx, 30xx	nvidia-open-dkms	Estándar. Rendimiento idéntico al propietario.
Turing	RTX 20xx, GTX 16xx	nvidia-open-dkms	Soportado. Requiere firmware GSP.
Pascal / Maxwell	GTX 10xx, GTX 9xx	nvidia-580xx-dkms (AUR)	Legacy. Soporte eliminado en drivers 590+.
Kepler y anteriores	GTX 7xx, 6xx	nvidia-470xx-dkms (AUR)	Obsoleto. Mantenimiento mínimo por comunidad.

El Peligro de la Actualización:

Si usted posee una tarjeta GTX 1080 Ti (Pascal) o anterior, una actualización estándar (pacman -Syu) en 2025 podría dejarle sin entorno gráfico, ya que los drivers oficiales nvidia ahora apuntan a la versión 590+ que no soporta su hardware. Debe migrar manualmente a los paquetes nvidia-580xx del AUR.

Instalación Paso a Paso para Hardware Moderno (Turing+):

1. **Limpieza Previa:** Elimine cualquier driver propietario antiguo para evitar conflictos.

Bash

```
sudo pacman -Rns nvidia nvidia-utils
```

2. Instalación del Stack Moderno:

Recomendamos la variante dkms para asegurar que los módulos se recompilen automáticamente con cada actualización del kernel (fundamental en una distribución Rolling Release como Arch).

Bash

```
sudo pacman -Syu nvidia-open-dkms nvidia-utils lib32-nvidia-utils opencl-nvidia cuda
```

Nota: El paquete cuda en los repositorios de Arch es generalmente la última versión estable (ej. 12.4 en 2025). PyTorch ha mejorado su compatibilidad, pero siempre verifique la matriz de compatibilidad.¹³

3. Configuración del Kernel (KMS para Wayland):

En 2025, Wayland es el protocolo de visualización por defecto en GNOME y KDE. Para que NVIDIA funcione correctamente, debe habilitar el Kernel Mode Setting (KMS) temprano.

Edite /etc/mkinitcpio.conf y añada los módulos a la lista MODULES:

Bash

```
MODULES=(... nvidia nvidia_modeset nvidia_uvm nvidia_drm...)
```

Luego, cree el archivo /etc/modprobe.d/nvidia.conf con el siguiente contenido para habilitar el modo DRM (necesario para la sincronización vertical y Wayland):

Bash

```
options nvidia_drm modeset=1 fbdev=1
```

Finalmente, regenere la imagen del kernel:

Bash

```
sudo mkinitcpio -P
```

3.2 Python y el PEP 668: La Muerte de sudo pip install

Uno de los cambios más disruptivos para los desarrolladores en 2024-2025 fue la adopción estricta del **PEP 668** ("Externally Managed Environments") por parte de las distribuciones Linux. Ahora, si intenta ejecutar pip install captum fuera de un entorno virtual, recibirá un

error bloqueante. Esto es una medida de seguridad para evitar que pip sobrescriba o rompa paquetes críticos del sistema gestionados por pacman.¹⁵

Estrategia de Gestión de Entornos para Deep Learning:

No utilice la bandera `--break-system-packages`. Es peligrosa y desaconsejada. En su lugar, adopte una estrategia híbrida que aproveche la potencia de los binarios precompilados de Arch y la flexibilidad de pip.

1. **La Base del Sistema:** Instale las bibliotecas pesadas (PyTorch, CUDA, Numpy) usando pacman. Los mantenedores de Arch (como Sven-Hendrik Haase) hacen un trabajo excepcional asegurando que `python-pytorch-cuda` esté compilado contra la versión exacta de CUDA del sistema, evitando bloatware y conflictos de bibliotecas dinámicas.

Bash

```
sudo pacman -S python python-pip python-virtualenv python-pytorch-cuda
torchvision-cuda python-numpy python-matplotlib
```

2. **Entornos Virtuales con Acceso al Sistema:** Cree un entorno virtual que tenga acceso a los paquetes instalados en el sistema (`--system-site-packages`). Esto le permite usar el PyTorch optimizado de Arch, pero instalar bibliotecas ligeras o específicas (como Captum) localmente.

Bash

```
# Crear el directorio del proyecto
```

```
mkdir ~/proyectos/xai_resnet && cd ~/proyectos/xai_resnet
```

```
# Crear entorno virtual que "ve" los paquetes globales (PyTorch, CUDA)
```

```
python -m venv --system-site-packages.venv
```

```
# Activar entorno
```

```
source.venv/bin/activate
```

```
# Instalar bibliotecas específicas de XAI (no disponibles en pacman)
```

```
pip install captum definition-free-xai
```

Esta configuración es robusta, respeta las normas de 2025 y ahorra espacio en disco al no duplicar gigabytes de binarios de PyTorch en cada proyecto.

4. Ecosistema de Herramientas 2025: La Hegemonía de Captum

Al analizar los recursos educativos y repositorios de código del último año, emerge un claro ganador en la guerra de las bibliotecas de interpretabilidad.

4.1 PyTorch & Captum: El Estándar de la Industria

Captum (latín para "comprensión") es la biblioteca oficial de interpretabilidad de PyTorch, desarrollada por Meta AI. En 2025, se destaca por varias razones ¹:

- **Diseño Unificado:** Ofrece una API polimórfica. Una vez que aprende a usar IntegratedGradients, puede cambiar a Saliency, DeepLIFT o NoiseTunnel cambiando una sola línea de código. Todos heredan de la clase base Attribution.
- **Soporte Multimodal:** No se limita a imágenes. Los tutoriales recientes muestran su eficacia en modelos de lenguaje (LLMs) y series temporales, tratando los tokens o pasos de tiempo como características de entrada diferenciables.
- **Métricas de Evaluación:** A diferencia de otras bibliotecas que solo generan visualizaciones, Captum incluye métricas cuantitativas como **Infidelidad** (Infidelity) y **Sensibilidad** (Sensitivity), permitiendo evaluar matemáticamente la calidad de la explicación.

4.2 El Estado de TensorFlow y Otras Alternativas

Para los usuarios de TensorFlow/Keras, el panorama es más fragmentado.

- **tf-explain:** Sigue siendo popular por su simplicidad y callbacks para TensorBoard, pero su desarrollo ha desacelerado.
- **TensorFlow Core:** Google recomienda implementar IG directamente usando tf.GradientTape para un control máximo, como se ve en sus tutoriales oficiales.¹⁸ Esto es excelente para el aprendizaje, pero verboso para la producción.
- **Alibi Explain:** Una biblioteca agnóstica que ha ganado tracción para datos tabulares y detección de desviaciones (drift), pero para visión pura en PyTorch, Captum sigue siendo superior.

Tabla 2: Comparativa de Herramientas de XAI (2025)

Característica	Captum (PyTorch)	tf-explain (Keras)	SHAP (Agnóstico)
Soporte IG	Nativo, Optimizado (C++)	Nativo	Aproximado (GradientExplainer)
Facilidad de Uso	Alta (API consistente)	Media (Callbacks)	Alta (pero lento para imágenes)
Ruido/Suavizado	Nativo (NoiseTunnel)	Manual	Kernel explainer (lento)
Métricas	Infidelidad/Sensibilidad	No	No
Comunidad 2025	Dominante	Decreciente	Estable (Fuerte en Tabular)

Dada la tendencia, este manual adoptará **Captum** como la herramienta vehicular para el proyecto práctico.

5. Manual Maestro: Proyecto Práctico "El Cirujano de Cajas Negras"

En esta sección, realizaremos una implementación de grado profesional de Gradientes Integrados. No nos limitaremos a generar una imagen bonita; validaremos la explicación y mitigaremos los errores comunes de ruido.

Objetivo del Proyecto: Diagnosticar las predicciones de un modelo ResNet50 pre-entrenado, identificando características críticas y evaluando la robustez de la explicación.

5.1 Fase 1: Preparación del Modelo y Datos

Primero, configuramos nuestro entorno e importamos las dependencias. Un detalle crítico en 2025 es el manejo de los pesos pre-entrenados; torchvision ha deprecado el parámetro pretrained=True en favor de la clase Weights.

Python

```
import torch
import torch.nn.functional as F
from torchvision import models, transforms
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from captum.attr import IntegratedGradients, NoiseTunnel
from captum.attr import visualization as viz

# Configuración de dispositivo agnóstica
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 1. Cargar el modelo
# Usamos la API moderna de pesos. IMAGENET1K_V1 garantiza reproducibilidad.
model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)

# CRÍTICO: Modo evaluación
# Si olvidamos esto, las capas de Dropout y BatchNorm seguirán activas,
# introduciendo estocasticidad que arruinará la explicación (gradientes ruidosos).
model = model.eval()
model = model.to(device)
```

5.2 Fase 2: Preprocesamiento y la Trampa de la Normalización

Uno de los errores más comunes documentados ⁵ es calcular los gradientes sobre la imagen original en lugar de la imagen normalizada que ve la red. IG debe operar en el espacio de entrada *real* del modelo.

Python

```
# Definimos las transformaciones estándar de ImageNet
transform = transforms.Compose()

# Definimos la normalización por separado para poder revertirla visualmente
normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                std=[0.229, 0.224, 0.225])

def get_image_tensor(image_path):
    img = Image.open(image_path).convert('RGB')
    transformed_img = transform(img)
    # Añadimos dimensión de batch: ->
    input_tensor = normalize(transformed_img).unsqueeze(0).to(device)
    return input_tensor, transformed_img

# Cargar una imagen de prueba (ej. un perro Labrador)
input_tensor, original_tensor = get_image_tensor("dog.jpg")
```

5.3 Fase 3: Implementación de Gradientes Integrados (Vanilla)

Implementaremos la versión básica de IG. Aquí, el parámetro `n_steps` es vital. Representa la granularidad de nuestra integral de Riemann. Un valor bajo (ej. 10) es rápido pero impreciso; un valor alto (ej. 200) es lento pero exacto.

Python

```
# Inicializar el algoritmo base
ig = IntegratedGradients(model)

# Realizar la predicción para obtener la clase objetivo
output = model(input_tensor)
```

```

target_class = torch.argmax(output, dim=1).item()
print(f"Clase predicha: {target_class}")

# Calcular atribuciones
# baseline=None implica una imagen negra (tensores de ceros), el estándar por defecto.
attributions_ig, delta = ig.attribute(
    input_tensor,
    target=target_class,
    n_steps=50,          # Compromiso inicial velocidad/precisión
    return_convergence_delta=True
)

print(f"Convergence Delta (Error de aproximación): {delta.item()}")

```

El Canario en la Mina: Convergence Delta

El valor delta devuelto es crucial. Mide la violación del Axioma de Completitud.⁷

- La teoría dice: $\sum \text{Atribuciones} = F(x) - F(\text{baseline})$.
- El código calcula: $\Delta = |\sum \text{Atribuciones} - (F(x) - F(\text{baseline}))|$.
- **Regla de Oro:** Si $\Delta > 0.05$ (o un 5% de la magnitud de la salida), la aproximación integral es mala. El modelo es demasiado no lineal en el camino elegido. **Solución:** Aumente n_steps progresivamente (50 -> 100 -> 200) hasta que el delta disminuya.

5.4 Fase 4: Refinamiento con Noise Tunnel (Suavizado)

En arquitecturas profundas como ResNet, los gradientes a menudo sufren del problema de "gradientes fracturados" (shattered gradients): variaciones violentas y de alta frecuencia entre píxeles vecinos que hacen que el mapa de calor parezca estática de TV.²⁰

Para solucionar esto, utilizamos la técnica **Noise Tunnel**. Esta técnica añade ruido gaussiano a la imagen de entrada varias veces, calcula IG para cada versión ruidosa y promedia los resultados. Es análogo a tomar múltiples exposiciones de una foto para reducir el grano.

Python

```

# Envolver IG dentro de Noise Tunnel
nt = NoiseTunnel(ig)

# Calcular atribuciones suavizadas (SmoothGrad)
# nt_samples: Número de imágenes ruidosas a promediar (10 es estándar, 50 es alta calidad)
# stdevs: Magnitud del ruido (0.1 - 0.2 es típico)
attributions_nt = nt.attribute(

```

```

input_tensor,
nt_type='smoothgrad',
nt_samples=10,
stdevs=0.15,
target=target_class,
n_steps=50
)

```

Este paso aumenta el tiempo de cómputo linealmente con `nt_samples`, pero la mejora en la legibilidad y enfoque del mapa de calor es dramática, eliminando el ruido de fondo irrelevante.

5.5 Fase 5: Visualización Profesional

Captum ofrece herramientas de visualización, pero para un informe profesional, necesitamos personalizarlas. Debemos transponer los tensores al formato que espera Matplotlib (Alto, Ancho, Canales) y des-normalizar para mostrar la imagen original correctamente.

Python

```

def visualize_attribution(attributions, original_img_tensor, title):
    # Transponer -> y mover a CPU
    attr_np = np.transpose(attributions.squeeze().cpu().detach().numpy(), (1, 2, 0))
    orig_np = np.transpose(original_img_tensor.squeeze().cpu().detach().numpy(), (1, 2, 0))

    # Visualizar usando utilidades de Captum
    _ = viz.visualize_image_attr(
        attr_np,
        orig_np,
        method="blended_heat_map", # Superpone el calor sobre la imagen
        sign="all",                 # Muestra contribuciones positivas (verde) y negativas (rojo)
        show_colorbar=True,
        title=title,
        outlier_perc=2              # Recorta el 2% superior para mejorar contraste
    )

visualize_attribution(attributions_ig, original_tensor, "IG Vanilla (Ruidoso)")
visualize_attribution(attributions_nt, original_tensor, "IG + SmoothGrad (Nítido)")

```

6. Conceptos Avanzados, Errores Comunes y Soluciones

Esta sección aborda las sutilezas que separan a un novato de un experto, basadas en problemas recurrentes en foros y literatura técnica.

6.1 El Dilema de la Línea Base: La Analogía del Cuarto Oscuro

La mayoría de los tutoriales usan una imagen negra (tensores de ceros) como línea base (x'). Esto asume que "negro" significa "ausencia de información".

El Problema: Imagine que entra a un cuarto oscuro. No ve nada. Si enciende una linterna, ve los objetos. Aquí, la oscuridad es una buena línea base. Pero, ¿qué pasa si el objeto que intenta clasificar es un oso negro en una cueva oscura? Para la red neuronal, el valor de píxel 0,0,0 (negro) es una característica definitoria del oso. Si su línea base también es negra, la diferencia $(x - x')$ es cero. Matemáticamente, el método dirá que el color del oso no contribuyó a la predicción, lo cual es falso.²²

Solución Avanzada:

1. **Línea Base Blanca o Ruido:** Pruebe con una imagen de ruido uniforme o blanca para contrastar.
2. **Múltiples Líneas Base:** Captum permite pasar un tensor de baselines. Lo ideal es usar el "promedio del dataset" o difuminar la imagen original (blurring) como línea base, eliminando la textura pero manteniendo el color promedio.

6.2 Errores de Forma (Shape Mismatch) y Modelos Hugging Face

Un error clásico es `RuntimeError: size mismatch`. Esto ocurre frecuentemente con modelos modernos (como los de Hugging Face transformers) que no devuelven un tensor simple, sino un objeto `ModelOutput` (una especie de diccionario). Captum espera un tensor para calcular gradientes.²⁴

Solución (Wrapper Function):

Debe envolver su modelo en una función que extraiga el tensor exacto.

Python

```
def forward_wrapper(inputs):
    output = model(inputs)
    # Si es un modelo de detección o HF que devuelve un diccionario
    if hasattr(output, 'logits'):
        return output.logits
    return output
```

```
# Al instanciar IG, pase el wrapper en lugar del modelo
ig = IntegratedGradients(forward_wrapper)
```

6.3 La Maldición de la Dimensión (Batch Processing)

En producción, explicar una imagen a la vez es ineficiente. IG requiere `n_steps` pasadas hacia adelante y hacia atrás. Para una imagen, con 50 pasos, son 50 pasadas.

Captum optimiza esto internamente procesando los pasos como un "batch". Sin embargo, si su GPU tiene poca VRAM (ej. 8GB), intentar procesar una imagen de alta resolución con 200 pasos puede causar un CUDA Out of Memory.

Estrategia: Use el parámetro `internal_batch_size` en el método `.attribute()`. Esto le dice a Captum que, aunque la integral requiera 200 pasos, solo procese 10 o 20 a la vez en la GPU, acumulando los resultados secuencialmente.

7. Conclusión y Futuro

La interpretabilidad de las redes neuronales ha madurado. En 2025, ya no dependemos de intuiciones visuales vagas; contamos con herramientas axiomáticas como los **Gradientes Integrados** que nos ofrecen garantías matemáticas de completitud y sensibilidad. Este manual ha recorrido desde la teoría fundamental —donde desmitificamos la integración de caminos mediante analogías de escalada y presupuestos— hasta la implementación práctica en un entorno Linux moderno y riguroso. Hemos visto que la elección de herramientas (Captum sobre implementaciones ad-hoc), la configuración del sistema (Drivers Open Kernel, Entornos Virtuales PEP 668) y la atención a los detalles finos (Convergence Delta, Noise Tunnel) son lo que diferencia una visualización engañosa de una explicación científica válida.

Mirando hacia el futuro, la investigación se dirige hacia **Guided Integrated Gradients (GIG)** y la adaptación de estos métodos a los Vision Transformers, donde la atención y los gradientes deben reconciliarse. Sin embargo, los principios aquí expuestos —la importancia de la línea base, la validación axiomática y el manejo del ruido— seguirán siendo los pilares sobre los que se construirá la confianza en la Inteligencia Artificial.

Referencias Citadas en el Texto:

5 (Teoría y Axiomas); 2 (Analogías); 11 (Drivers NVIDIA/Arch); 15 (Python PEP 668); 1 (Captum vs otros); 7 (Completeness/Delta); 20 (Noise Tunnel/Shattered Gradients); 22 (Problema de Baseline); 24 (Errores de Forma).

Obras citadas

1. meta-pytorch/captum: Model interpretability and understanding for PyTorch - GitHub, fecha de acceso: enero 7, 2026, <https://github.com/meta-pytorch/captum>
2. The Mountain Climbing Analogy by Naval Ravikant | #shorts #naval #joerogan - YouTube, fecha de acceso: enero 7, 2026, <https://www.youtube.com/shorts/Ko0MWv5TZ5s>
3. The Mountain in the Machine: Optimization and the Landscapes of Machine Learning – Sam P. Kellogg, fecha de acceso: enero 7, 2026, <https://culturemachine.net/vol-20-machine-intelligences/the-mountain-in-the-machine-optimization-and-the-landscapes-of-machine-learning-sam-p-kellogg/>
4. Hill climbing - Wikipedia, fecha de acceso: enero 7, 2026, https://en.wikipedia.org/wiki/Hill_climbing
5. Integrated Gradients: A Comprehensive Guide for 2025 - Shadecoder - 100% Invisible AI Coding Interview Copilot, fecha de acceso: enero 7, 2026, <https://www.shadecoder.com/topics/integrated-gradients-a-comprehensive-guide-for-2025>
6. Interpreting Deep Neural Networks using Integrated Gradients - Towards Data Science, fecha de acceso: enero 7, 2026, <https://towardsdatascience.com/interpreting-deep-neural-networks-using-integrated-gradients-f9b8ecdd3c57/>
7. Integrated Gradients is a Nonlinear Generalization of the Industry Standard Approach to Variable Attribution for Credit Risk Models - Digital Commons@Kennesaw State, fecha de acceso: enero 7, 2026, <https://digitalcommons.kennesaw.edu/cgi/viewcontent.cgi?article=1030&context=dataphdgreylit>
8. Limitations of Integrated Gradients for Feature Attribution | by Elan Stopnitzky - Medium, fecha de acceso: enero 7, 2026, <https://medium.com/data-science/limitations-of-integrated-gradients-for-feature-attribution-ca2a50e7d269>
9. Explaining Machine Learning Models - Stanford CS Theory, fecha de acceso: enero 7, 2026, <http://theory.stanford.edu/~ataly/Talks/ExplainingMLModels.pdf>
10. A New Baseline Assumption of Integrated Gradients Based on Shaply value - arXiv, fecha de acceso: enero 7, 2026, <https://arxiv.org/html/2310.04821v3>
11. NVIDIA 590 driver drops Pascal and lower support; main packages switch to Open Kernel Modules - Arch Linux, fecha de acceso: enero 7, 2026, <https://archlinux.org/news/nvidia-590-driver-drops-pascal-support-main-packages-switch-to-open-kernel-modules/>
12. Arch Linux's Main NVIDIA Driver Packages Now Using The Open Kernel Modules - Phoronix, fecha de acceso: enero 7, 2026, <https://www.phoronix.com/news/Arch-Linux-NVIDIA-Open-Default>
13. Install PyTorch on Windows, Linux and macOS in 2025: Step by Step - Hugging Face, fecha de acceso: enero 7, 2026, <https://huggingface.co/blog/daya-shankar/pytorch-install-guide>

14. python-pytorch-cuda 2.9.1-6 (x86_64) - Arch Linux, fecha de acceso: enero 7, 2026, https://archlinux.org/packages/extra/x86_64/python-pytorch-cuda/
15. pip install -r requirements.txt is failing: "This environment is externally managed" [duplicate], fecha de acceso: enero 7, 2026, <https://stackoverflow.com/questions/75602063/pip-install-r-requirements-txt-is-failing-this-environment-is-externally-managed>
16. PEP 668 – Marking Python base environments as “externally managed”, fecha de acceso: enero 7, 2026, <https://peps.python.org/pep-0668/>
17. Using Captum for Interpreting PyTorch Models in Production | by Nivedhitha D | Medium, fecha de acceso: enero 7, 2026, <https://medium.com/@nivedhithadm/using-captum-for-interpreting-pytorch-models-in-production-ddf91fbbd3fd>
18. Integrated gradients | TensorFlow Core, fecha de acceso: enero 7, 2026, https://www.tensorflow.org/tutorials/interpretability/integrated_gradients
19. fecha de acceso: enero 7, 2026, <https://medium.com/@nivedhithadm/using-captum-for-interpreting-pytorch-models-in-production-ddf91fbbd3fd#:~:text=Meaning%3A%20The%20convergence%20delta%20measures,model's%20prediction%20and%20baseline%20output.>
20. Guided Integrated Gradients: An Adaptive Path Method for Removing Noise - CVF Open Access, fecha de acceso: enero 7, 2026, https://openaccess.thecvf.com/content/CVPR2021/papers/Kapishnikov_Guided_Integrated_Gradients_An_Adaptive_Path_Method_for_Removing_Noise_CVPR_2021_paper.pdf
21. Gradient based Feature Attribution in Explainable AI: A Technical Review - arXiv, fecha de acceso: enero 7, 2026, <https://arxiv.org/html/2403.10415v1>
22. XAI Methods — Integrated Gradients | by Kemal Erdem (burnpiro) - Medium, fecha de acceso: enero 7, 2026, <https://medium.com/@kemalpiro/xai-methods-integrated-gradients-6ee1fe4120d8>
23. Deep Dive into Neural Network Explanations with Integrated Gradients, fecha de acceso: enero 7, 2026, <https://towardsdatascience.com/deep-dive-into-neural-network-explanations-with-integrated-gradients-7f4b7be855a2/>
24. Normal inference works fine, but captum (IntegratedGradients) throws a size mismatch, fecha de acceso: enero 7, 2026, <https://discuss.pytorch.org/t/normal-inference-works-fine-but-captum-integratedgradients-throws-a-size-mismatch/101378>
25. Unable to use LayerIntegratedGradients when embedding multiple sentences via BERT at once · Issue #329 · meta-pytorch/captum - GitHub, fecha de acceso: enero 7, 2026, <https://github.com/pytorch/captum/issues/329>
26. AssertionError: Attributions tensor and the end_point must match on the first dimension but found attribution: 4 and end_point: 1 · Issue #457 · meta-pytorch/captum - GitHub, fecha de acceso: enero 7, 2026, <https://github.com/pytorch/captum/issues/457>